THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science · Faculty of Science and Technology

LIMO Instruction Set Architecture

Milestone M1 Submission

CS3520 Computer Organisation and Architecture · LIMO Group Project · AY 2026/2027

Team	limo-nul	**Release tag**	m1
Lab group	C	**Commit**	[short SHA]
Due	Sun 4 Oct 2026, 23:59	**Page limit**	6 pages

#	Member	Student no.	GitHub
1	LEREKO THULO	202322270	recocipher-beep
2	QABA MOIKETSI	202002573	qabamothe
3	TUMELO BERNARD LEPUTLA	202403700	TumeloLeputla2006
4	SETENTE NTHAKO	202321761	Setente
5	THATO LEPITLA	202201975	Thatosh
6	TS'EBETSO JADEN LEKHANYA	202322765	jadenlekhanya
7			

Assessment. ISA specification A1–A5 (40 marks), design-decision log A6 (20 marks), and an individual defence at D1 (40 marks).

1 Team Charter and Set-up

Project board	https://github.com/recocipher-beep/limo-nul.git
Meeting rhythm	[stand-up FRI & SUN 17:00 - 21:00]
Decision rule	We use the "Majority rules" system
Definition of done	merged through a reviewed PR, tests pass, issue closed
Branch and PR rules	SETENTE
When a member is absent or not contributing	We issue the member a verbal notice and try to contact them

2 ISA Specification

This section uses the course ISA specification template. Each design choice is described first, followed by the reason for choosing it and the main trade-offs.

2.1 Overview

	Specification	Rationale
Architecture style	32-bit load–store with fixed 32-bit instructions; only `tsl` and `bl` access memory.	Most instructions operate on registers. The EX stage handles arithmetic and address calculations, while loads and stores access memory in MEM. The trade-off is that a memory operand usually needs a separate load or store. A CISC-style memory-operand design was not selected because variable-length instructions would complicate decoding and pipeline design.
Register bank	32 registers R0–R31 (5-bit fields). R0–R4 are SPRs (zero, return address, stack pointer, global pointer, thread pointer); R5–R31 are GPRs. The PC is separate.	Keeping 32 registers preserves the five-bit RV32I register fields and the associated calling-convention roles, making it easier to cross-check programs in Ripes. The trade-off is 1,024 bits of register storage and wider register comparisons. A 16- or 8-register design would save hardware, but the current instruction formats would not gain a useful number of extra bits.
Addressing modes	Register; immediate (12-bit signed); base + offset (`tsl`, `bl`, `tlkr`); PC-relative (branches, `tlk`, `tl`).	Register, signed 12-bit immediate, base-plus-offset and PC-relative addressing cover the current instruction set with the ALU and one adder. Indexed and auto-increment addressing were left out because they would require additional address-generation hardware or more register-port support.
Memory, endianness, alignment	Byte-addressed, little-endian, word-aligned; [memory size].	Byte addressing and little-endian ordering match RV32I and make memory contents easier to compare with Ripes. Word alignment keeps each word transfer straightforward in the MEM stage. The trade-off is that unaligned word accesses are not supported.
Instruction formats	R, I, S, B and J (J is used by `tlk` and `tl`).	The R, I, S and B formats cover arithmetic, logic, memory access and conditional branches. J format supports the longer reach required by `tlk` and `tl`. U format was omitted because the current instruction set has no load-upper-immediate or add-upper-immediate instruction.
Departures from RV32I	No change to field positions, opcodes or funct values of the 20 instructions kept. Departures: (1) a 20-instruction subset, with no lui, auipc, set-less-than, arithmetic or immediate shifts, byte or halfword accesses, fence, ecall or ebreak; (2) Sesotho mnemonics and register names; (3) shifts take the amount from a register only.	The handout limits the ISA to 10–20 instructions and excludes CSRs and exceptions. Keeping every encoding unchanged lets us cross-check each program in Ripes.

2.2 Register File

Number	Sesotho name	Alias	Type	Purpose and convention
R0	Lefela	`zero`	**SPR**	Hard-wired to 0; writes are ignored
R1	Aterese ea Khutlo	`ra`	**SPR**	Return address: where the program returns after a call
R2	Sesupa Pokello	`sp`	**SPR**	Stack pointer: current position of the stack in memory
R3	Sesupa Kakaretso	`gp`	**SPR**	Global pointer: area holding the program's general data
R4	Sesupa Mola	`tp`	**SPR**	Thread pointer: data of the current thread of execution
R5–R7	Nakoana 1–3	`t0–t2`	**GPR**	Temporaries; not preserved across calls
R8–R9	Bolokiloeng 1–2	`s0–s1`	**GPR**	Saved; must be preserved across calls
R10–R17	Khang 1–8	`a0–a7`	**GPR**	Arguments passed to a function and values it returns
R18–R27	Bolokiloeng 3–12	`s2–s11`	**GPR**	Saved; must be preserved across calls

Number	Sesotho name	Alias	Type	Purpose and convention
R28–R31	Nakoana 4–7	`t3–t6`	**GPR**	Temporaries; not preserved across calls
PC	Sesupa Taelo	—	**SPR**	Address of the current instruction; not in the register file

2.3 Instruction Formats

The design retains the RV32I field widths. Since LIMO has 32 registers, each register field requires 5 bits; reducing the number of registers would not create a useful benefit for the formats used here.

Bit	31 … 25	24 … 20	19 … 15	14 … 12	11 … 7	6 … 0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Immediate ranges

Format	Immediate	Range
I	12-bit signed	−2048 to +2047
S	12-bit signed	−2048 to +2047
B	13-bit signed, branch reach in bytes	−4096 to +4094 bytes, in steps of 2
J	21-bit signed, jump reach in bytes	−1,048,576 to +1,048,574 bytes, in steps of 2

Opcode and function-field assignments (funct3, plus funct7 for R-type)

Opcode	Fmt	Instructions
`0110011`	**R**	`kpn 000/0000000` · `tls 000/0100000` · `le 111/0000000` · `kp 110/0000000` · `kpe 100/0000000` · `stts 001/0000000` · `stto 101/0000000`
`0010011`	**I**	`kpnh 000` · `kph 110` · `leh 111` · `kpeh 100`
`0000011`	**I**	`tsl 010`
`0100011`	**S**	`bl 010`
`1100011`	**B**	`khl 000` · `khsl 001` · `khn 100` · `khk 101`
`1100111`	**I**	`tlkr 000`
`1101111`	**J**	`tlk` (rd receives the return address) · `tl` (rd = R0, return address discarded)

2.4 Instruction Set and Sesotho Mnemonics

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	`kpn`	*Kopanya*; add	`kpn rd, rs1, rs2`	R[rd] ← R[rs1] + R[rs2]	R	`add`
Arithmetic	`tls`	*Tlosa*; subtract	`tls rd, rs1, rs2`	R[rd] ← R[rs1] − R[rs2]	R	`sub`
Arithmetic	`kpnh`	*Kopanya hang-hang*; add immediate	`kpnh rd, rs1, imm`	R[rd] ← R[rs1] + SignExt(imm12)	I	`addi`
Logic	`le`	*Le*; AND	`le rd, rs1, rs2`	R[rd] ← R[rs1] AND R[rs2]	R	`and`

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Logic	`kp`	*Kapa*; OR	`kp rd, rs1, rs2`	R[rd] ← R[rs1] OR R[rs2]	R	`or`
Logic	`kpe`	*Kapa e khethehileng*; XOR	`kpe rd, rs1, rs2`	R[rd] ← R[rs1] XOR R[rs2]	R	`xor`
Logic	`leh`	*Le hang-hang*; AND immediate	`leh rd, rs1, imm`	R[rd] ← R[rs1] AND SignExt(imm12)	I	`andi`
Logic	`kph`	*Kapa hang-hang*; OR immediate	`kph rd, rs1, imm`	R[rd] ← R[rs1] OR SignExt(imm12)	I	`ori`
Logic	`kpeh`	*Kapa e khethehileng hang-hang*; XOR immediate	`kpeh rd, rs1, imm`	R[rd] ← R[rs1] XOR SignExt(imm12)	I	`xori`
Shift	`stts`	*Suthisa ho le letšehali*; shift left	`stts rd, rs1, rs2`	R[rd] ← R[rs1] << R[rs2][4:0]	R	`sll`
Shift	`stto`	*Suthisa ho le letona*; shift right (logical)	`stto rd, rs1, rs2`	R[rd] ← R[rs1] >> R[rs2][4:0]	R	`srl`
Memory	`tsl`	*Ts'ela lentsoe*; load word	`tsl rd, imm(rs1)`	R[rd] ← M[R[rs1] + SignExt(imm12)]	I	`lw`
Memory	`bl`	*Boloka lentsoe*; store word	`bl rs2, imm(rs1)`	M[R[rs1] + SignExt(imm12)] ← R[rs2]	S	`sw`
Jump	`tlk`	*Tlola u khutla*; jump and link	`tlk rd, label`	R[rd] ← PC + 4; PC ← PC + SignExt(offset21)	J	`jal`
Jump	`tlkr`	*Tlola u khutla ka rejistara*; jump and link register	`tlkr rd, imm(rs1)`	R[rd] ← PC + 4; PC ← (R[rs1] + SignExt(imm12)) with bit 0 cleared	I	`jalr`
Jump	`tl`	*Tlola*; jump (tlk with rd = R0)	`tl label`	PC ← PC + SignExt(offset21)	J	`jal x0, (j)`
Branch	`khl`	*Kheloha ha li lekana*; branch if equal	`khl rs1, rs2, label`	if R[rs1] = R[rs2] then PC ← PC + SignExt(offset13)	B	`beq`
Branch	`khsl`	*Kheloha ha li sa lekane*; branch if not equal	`khsl rs1, rs2, label`	if R[rs1] ≠ R[rs2] then PC ← PC + SignExt(offset13)	B	`bne`
Branch	`khn`	*Kheloha ha e le nyane*; branch if less (signed)	`khn rs1, rs2, label`	if R[rs1] < R[rs2] then PC ← PC + SignExt(offset13)	B	`blt`
Branch	`khk`	*Kheloha ha e le kholo*; branch if greater or equal (signed)	`khk rs1, rs2, label`	if R[rs1] ≥ R[rs2] then PC ← PC + SignExt(offset13)	B	`bge`

Lexical rules

Mnemonics	Plain letters, for example `kpn`, `tsl` and `khk`.
Comments	A semicolon begins a comment that continues to the end of the line.
Labels	A name followed by a colon, such as `LOOP:`. Branch and jump instructions use the label as their target.
Registers	Written as `R0`–`R31`.
Memory operands	`offset(base)` notation, for example `0(R10)`.
Immediates	Written in decimal.
Apostrophes	In full Sesotho terms, an apostrophe is part of the spelling rather than a separator. The assembler therefore treats straight and curly apostrophes as equivalent and must not split a word at an apostrophe. The mnemonics themselves contain none; for example, "Ts'ela lentsoe" is represented by `tsl`.

2.5 Encoding Worked Examples

One instruction per format, encoded by hand. The assembler must produce exactly these bits.

Instruction	Fmt	Field values	Binary (bit 31 … 0)	Hex
`kpn R7, R5, R6`	R	funct7 0000000, rs2 00110, rs1 00101, funct3 000, rd 00111, opcode 0110011	`0000000 00110 00101 000 00111 0110011`	`0x006283B3`
`kpnh R5, R0, 1`	I	imm 000000000001, rs1 00000, funct3 000, rd 00101, opcode 0010011	`000000000001 00000 000 00101 0010011`	`0x00100293`
`bl R6, 4(R10)`	S	imm[11:5] 0000000, rs2 00110, rs1 01010, funct3 010, imm[4:0] 00100, opcode 0100011	`0000000 00110 01010 010 00100 0100011`	`0x00652223`
`khk R5, R7, +8`	B	imm[12] 0, imm[10:5] 000000, rs2 00111, rs1 00101, funct3 101, imm[4:1] 0100, imm[11] 0, opcode 1100011	`0 000000 00111 00101 101 0100 0 1100011`	`0x0072D463`
`tl −12`	J	imm[20] 1, imm[10:1] 1111111010, imm[11] 1, imm[19:12] 11111111, rd 00000, opcode 1101111	`1 1111111010 1 11111111 00000 1101111`	`0xFF5FF06F`

3 Design-Decision Log

This section records the main design choices, the alternatives considered and the reasons for accepting the trade-offs. Each answer is at most 120 words and argues from the team's own programs.

(a) Register-file size 113 words

A 32-register file needs 5 bits per register field, so three fields take 15 of the 32 bits in an R-type instruction. The file holds 32 × 32 = 1,024 bits and needs two 32-to-1 read multiplexers; the forwarding and load-use comparators also work on 5-bit identifiers. The main benefit is compatibility with the RV32I encodings LIMO retains, so programs can be checked in Ripes. It also leaves enough registers for arguments, saved values and temporaries without frequent spills. Program 3, the largest sample, uses only seven working registers (R5–R11), so the upper registers are not exercised by the examples. The team accepts the extra hardware cost for compatibility and room to grow.

(b) Architectural or microarchitectural? 95 words

The program-visible state is the PC, registers R0–R31 and data memory. The instruction register, the pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB), control signals, forwarding selections and bubble flags belong to the implementation and must not affect the meaning of a program. The ISA must not specify them, because doing so would tie every implementation to one pipeline. A correct single-cycle or pipelined design therefore produces the same final state; only timing differs. In Program 1 the final value is R6 = 55, but the two designs may take different numbers of cycles to reach it.

(c) Branch resolution 120 words

LIMO resolves branches and jumps in the EX stage and predicts not taken. A taken transfer flushes the instructions in IF and ID, a two-cycle penalty. In Program 1, ten taken transfers (nine `tl` jumps and the final khk) cost 20 of the 66 cycles: 42 executed instructions, four pipeline-fill cycles and 20 flushed cycles. Resolving in ID would cut each flush to one instruction, but needs comparison hardware, forwarding into ID, and a stall when a branch follows the instruction producing its operand. Here khk follows kpnh R5 every pass, so ID resolution would cost 42 + 4 + 10 flushes + 10 stalls = 66 cycles, no better than EX. EX reuses the ALU and existing forwarding paths.

(d) Load-use 110 words

Forwarding can send a result from a pipeline register to an ALU input, but a load's data is not available until the end of MEM. In Program 3, `tsl R6, 0(R11)` reads memory in cycle 4 of the diagram below, while the next instruction, `kpn R7, R5, R6`, needs R6 at the start of its EX stage in that same cycle. The processor inserts one stall and forwards the loaded value from MEM/WB to EX in cycle 5. Program 3 therefore takes 16 cycles for 11 instructions. The hazard unit detects the case when ID/EX.MemRead is set and the destination register matches rs1 or rs2 of the instruction in ID.

Clock cycle	1	2	3	4	5	6	7
`tsl R6, 0(R11)`	IF	ID	EX	MEM	WB		
`kpn R7, R5, R6`		IF	ID	ID (stall)	EX (fwd)	MEM	WB

Orange: the one-cycle load-use stall. Green: the loaded value is forwarded from MEM/WB to the EX input in cycle 5.

(e) A flags register? 91 words

LIMO's branch instructions compare two registers directly (`khl`, `khsl`, `khn` and `khk`). A separate flags register would not remove an instruction from the loops in Programs 1 and 2, because their final branches already test the counter. It would also add state and dependencies: arithmetic instructions that set flags become producers, branches depend on those flags, and extra forwarding is needed. The flags would also have to be preserved across calls. Given the limited encoding space, a flags register would add complexity without reducing the instruction count or improving these examples.

(f) What breaks first as programs grow? 96 words

The immediate range is likely to become restrictive first. Without a load-upper-immediate instruction, constants outside −2048 to +2047 need several instructions and a temporary register; constructing 4096 takes three instructions in the current subset rather than one. Branch reach is next: a B-type branch covers roughly 4 KiB either way, about 1,024 instructions each direction, so larger programs may need a branch around a jump. Register pressure is the least urgent, since Program 3 uses only seven registers. If the ISA is extended, a load-upper-immediate instruction such as `lui` would be the most useful first addition.

4 Sample LIMO Programs

The cycle estimates assume the five-stage pipeline from the design-decision log: full forwarding, branch and jump resolution in EX, a not-taken prediction, a two-cycle penalty for each taken transfer, and a one-cycle stall for a load-use hazard.

Program 1 Sum from 1 to 10

This program uses a counter and an accumulator. It adds the counter to the running sum, increments the counter, and repeats until the counter reaches 11.

```
1   kpnh R5, R0, 1              ; R5 = 1 (counter)
2   kpnh R6, R0, 0              ; R6 = 0 (sum)
3   kpnh R7, R0, 11             ; R7 = 11 (one past the last number)
4   LOOP:
5   kpn  R6, R6, R5             ; sum = sum + R5
6   kpnh R5, R5, 1              ; R5 = R5 + 1
7   khk  R5, R7, DONE           ; if R5 >= 11, finish
8   tl   LOOP                   ; repeat
9   DONE:
```

Expected final state	Instructions executed	Estimated cycles
R6 = 55, R5 = 11, R7 = 11	42	66 (10 taken transfers)

Program 2 Countdown

This program starts R5 at 10 and subtracts 1 on each pass through the loop. It stops when R5 reaches zero.

```
1   kpnh R5, R0, 10             ; R5 = 10
2   kpnh R6, R0, 1              ; R6 = 1
3   LOOP:
4   tls  R5, R5, R6             ; R5 = R5 - 1
5   khl  R5, R0, DONE           ; if R5 == 0, finish
6   tl   LOOP                   ; repeat
7   DONE:
```

Expected final state	Instructions executed	Estimated cycles
R5 = 0, R6 = 1	31	55 (10 taken transfers)

Program 3 Add two values in memory, then mask and shift

This program stores 20 and 30 at addresses 100 and 104, loads both values, adds them, applies a bit mask and shifts the result.

```
1   kpnh R10, R0, 100          ; R10 = 100 (address of first word)
2   kpnh R5, R0, 20            ; R5 = 20
3   bl   R5, 0(R10)            ; Memory[100] = 20
4
5   kpnh R6, R0, 30            ; R6 = 30
6   kpnh R11, R10, 4           ; R11 = 104 (address of second word)
7   bl   R6, 0(R11)            ; Memory[104] = 30
8
9   tsl  R5, 0(R10)            ; R5 = Memory[100] = 20
10  tsl  R6, 0(R11)            ; R6 = Memory[104] = 30
11  kpn  R7, R5, R6            ; R7 = 50 (load-use stall here)
12
13  leh  R8, R7, 15            ; R8 = 50 AND 15 = 2
14  stts R9, R7, R8            ; R9 = 50 << 2 = 200
```

Expected final state	Instructions executed	Estimated cycles
Memory[100] = 20, Memory[104] = 30; R5 = 20, R6 = 30, R7 = 50, R8 = 2, R9 = 200, R10 = 100, R11 = 104	11	16 (incl. one load-use stall)

5 References

- Patterson, D. A. and Hennessy, J. L. *Computer Organization and Design: The RISC-V Edition*, Chapters 2 and 4.
- Harris, S. L. and Harris, D. *Digital Design and Computer Architecture: RISC-V Edition*, Chapters 6–7.
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA* (RV32I).
- [Sesotho sources or speakers consulted for the mnemonics]

6 AI-Use Declaration

Tool	Used for	How we checked or changed the output
Claude	Presentation, Paraphrasing, Research	[We cross-referenced our output with our notes and our research]
ChatGPT	[Research]	[we were satisfied with the output]

We confirm that every member has reviewed the submission and can explain the ISA design, instruction encodings, design decisions and sample programs.

Sprint Note *maximum one page; not counted in the page limit*

Sprint goal: Finalize instruction set, register naming, and design decisions so coding can begin.

Member	Role this sprint	Delivered (issue and PR numbers)	PRs reviewed
LEREKO THULO	[Team Charter & PDF]	Issue #3	[#PR]
QABA MOIKETSI	[Machine Style, Binary Encoding]	[A1 and A6 | issue #6…]	None
TUMELO BERNARD LEPUTLA	[Sesotho Assembly Glossary]	[A6 and A4 | issue #4]	None
THATO LEPITLA	[Naming Registers, Instruction set]	[A2 and A3 | issue #5]	[issue #7 and issue #8]
TS'EBETSO JADEN LEKHANYA	[Sample Programs]	[Add LIMO sample programs issue #7]	[None…]
SETENTE NTHAKO	[GitHub Repository setup]	[Repository setup | issue #2…]	[#5, #4, #3]

Not done, and why	M2, M3-Their time has not arrived, it is a step-by-step project done in milestones
Next sprint	M2
Blockers	[Internet connection]